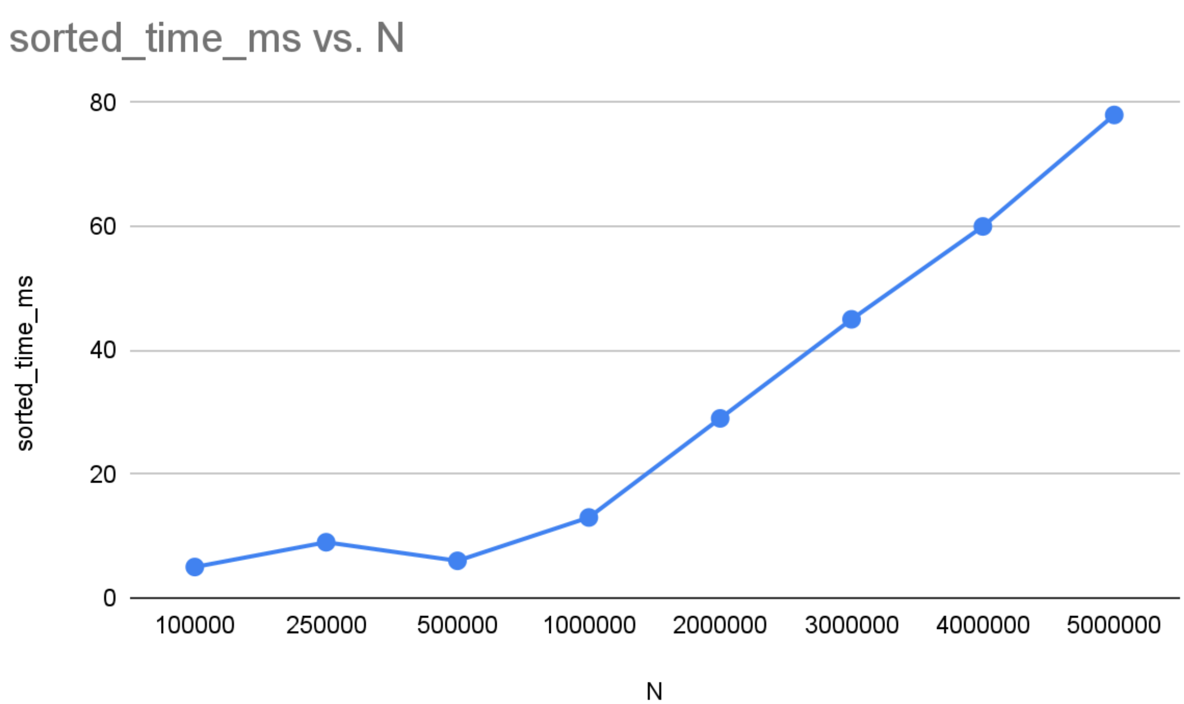
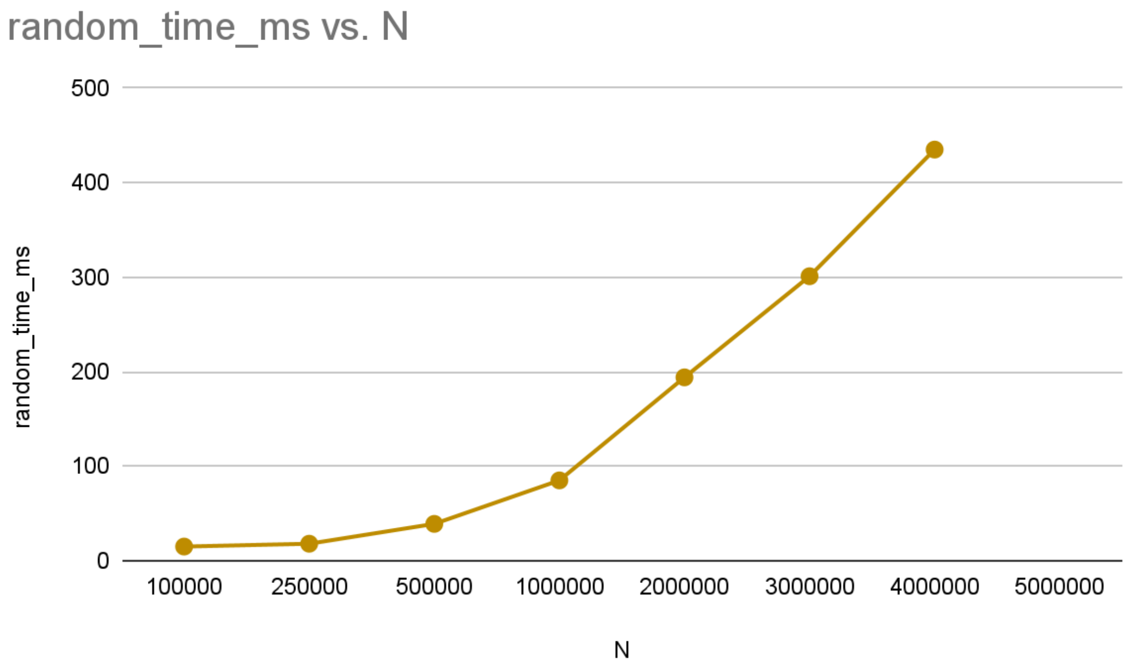
I used the quicksort implementation (median-of-three pivot and insertion sort) to measure running time on integer arrays of a few different sizes. I tested nine values of N ranging from 100,000 to 5,000,000 data elements and I utilized three input orderings for each value to compare the time complexity.

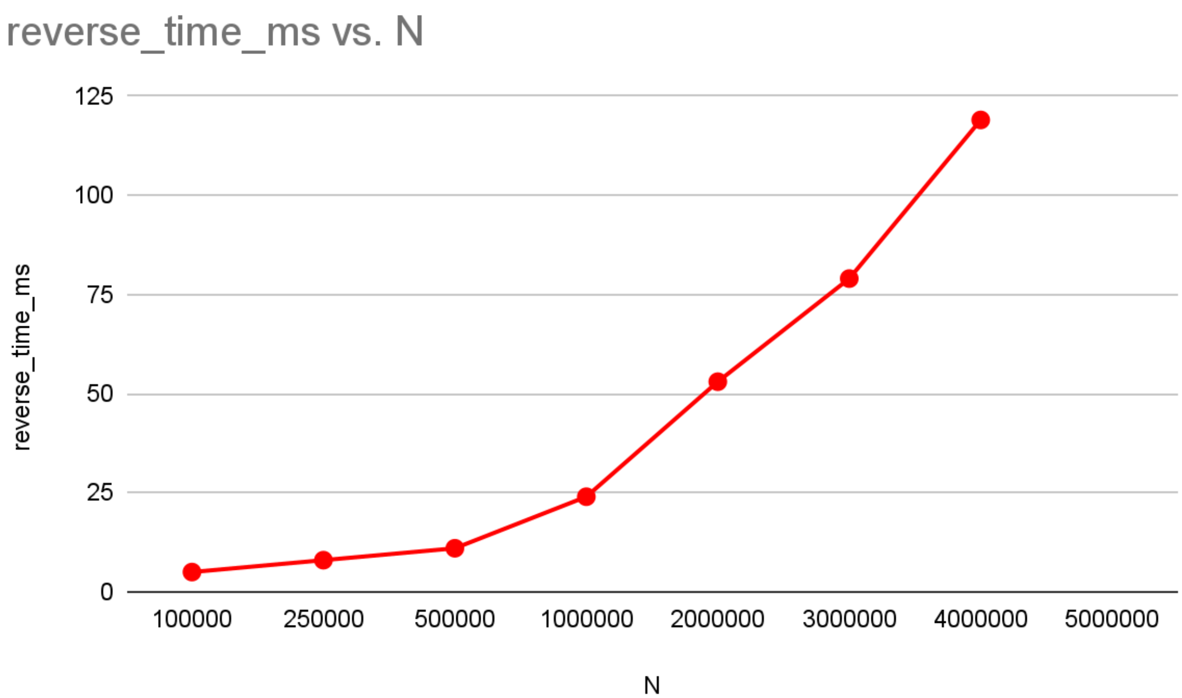
1. Ascending order
2. Descending (reverse) order
3. Randomly shuffled and picked

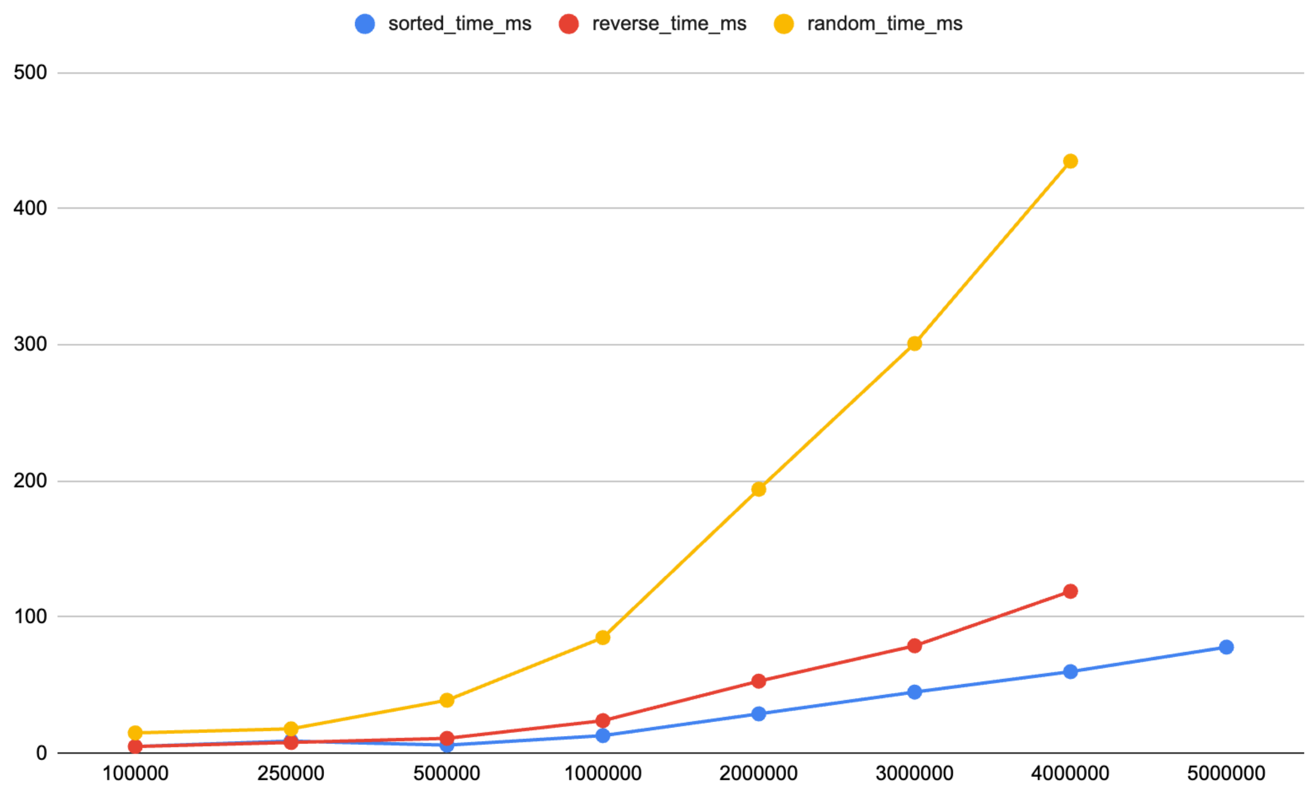
By choosing and testing 9 different values of N between 100,000 and 5,000,000, I was able to capture the abrupt change in time complexity as the number grew based on the ordering used. I also ran the program with a few other applications open.

Before running my experiment, I analyzed expected behavior from each of the orderings, where random input behaves in Tavg O(N log N), while sorted & reverse has Twrst O(N^2) behavior. The differences between the three graphs, specifically in the last graph where all three are combined, show these expected behaviors.




random_time_ms vs. N
500
400
300
200
100
0
random_time_ms
100000
250000
500000
1000000
2000000
3000000
4000000
5000000
N


reverse_time_ms vs. N
125
100
75
50
25
0
reverse_time_ms
100000
250000
500000
1000000
2000000
3000000
4000000
5000000
N



Overall, the running time increases as N increases for each of the three input types. For example in random input, N = 100,000 took about 15ms, meanwhile N = 2,000,000 for the same ordering takes 184ms. So as N doubles in size, the time grows faster by a factor slightly larger than 2, which goes to show O(N log N) behavior not O(N^2). The random curve increased smoothly as N gets larger and grows to match N log N. Even though the random-input timing line is higher than the sorted and reverse-sorted lines in some of my measurements, random ordering is still faster for Quicksort.

The sorted and reverse-sorted are the slowest, but didn't take as long as they could have. In my tests, this was because my code used median-of-three to increase efficiency. The curves for these two sorts grow faster at higher values of N. This shows the less balanced partitions and makes runtime closer to N^2 worst case.

Sorted and Reverse are Big O, Little O, Big Theta, and Big Omega to one another. They are also both little o to reverse. Reverse will always be Big Omega to the others.